

VideoAnnotator and Video Annotation Viewer: an open toolkit for automated multi-modal video annotation and cross-modal audit in behavioral research

Caspar Addyman¹✉, Jeremiah Ishaya¹, Irene Uwerikowe¹, Daniel Stamate², Jamie Lachman³, and Mark Tomlinson¹

¹ Institute for Life Course Health Research (ILCHR), Stellenbosch University, South Africa ² Department of Computing, Goldsmiths, University of London, United Kingdom ³ Department of Social Policy and Intervention (DISP), University of Oxford, United Kingdom ✉ Corresponding author

DOI: 10.xxxxxx/draft

Software

- [Review](#) ✉
- [Repository](#) ✉
- [Archive](#) ✉

Editor: ✉

Submitted: 04 March 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

We present two companion open-source tools for reproducible video-based behavioral research: **VideoAnnotator**, a Python pipeline toolkit, and **Video Annotation Viewer** (VAV), a browser-based audit interface. In this integrated workflow, VideoAnnotator is the processing engine and VAV is the review surface. Together they cover the full analysis loop, from raw video to cross-modal inspection of automated outputs, while keeping all processing local to meet data-governance requirements common in research involving children or other vulnerable populations.

VideoAnnotator provides ten declaratively configured pipeline specifications spanning four modalities: person tracking via YOLOv11 with ByteTrack (Jocher & Qiu, 2024; Zhang et al., 2022); facial analysis using DeepFace (Serengil & Ozpinar, 2020), LAION EmoNet face emotion (Schuhmann, Kaczmarczyk, Rabby, Kraus, et al., 2025), and OpenFace 3 (Hu et al., 2025); scene detection with PySceneDetect and CLIP-based labelling (Castellano, 2024; Radford et al., 2021); and audio processing comprising Whisper speech recognition (Radford et al., 2023), pyannote speaker diarization (Bredin & Laurent, 2023), and LAION EmoNet voice emotion (Schuhmann, Kaczmarczyk, Rabby, Friedrich, et al., 2025). Pipelines share a uniform interface behind a local-first FastAPI service (Ramírez, 2018) with Docker images for reproducible CPU and GPU execution. Outputs are standardized to established formats (COCO JSON, RTTM, WebVTT) and wrapped with provenance metadata recording the pipeline version and configuration.

Video Annotation Viewer is a React/TypeScript single-page application that renders those outputs both as overlays on the source video and as a synchronized, multi-track timeline beneath the source video. Each annotation stream (pose skeletons, speech captions, speaker diarization, scene boundaries) appears as a parallel, time-aligned lane, allowing researchers to audit cross-modal relationships at a glance: where a speaker label disagrees with who is on screen, where pose tracking drops out, or where turn-taking patterns emerge in caregiver-child interaction. The viewer reads standard files directly (COCO JSON, WebVTT, RTTM), and can also connect to a running VideoAnnotator service for job submission and results retrieval. Standalone use requires no server.

Statement of need

Behavioral and interaction research depends on observational video, yet human annotation is costly, slow, and difficult to reproduce across sites (Bakeman & Gottman, 1997). Automated pipelines can scale, but they introduce a new problem: researchers must be able to *inspect and trust* what each detector produced before drawing scientific conclusions, especially when constructs are subjective, context-dependent, or concern vulnerable populations.

The current landscape makes this harder than it should be. Each upstream model — face detectors, speech recognizers, pose estimators — has its own installation procedure, input expectations, and output format, so building a multi-modal analysis workflow requires substantial per-lab engineering. Once outputs exist, there is no lightweight tool for reviewing heterogeneous annotation streams together in time: seeing whether the transcribed speech matches the on-screen speaker, or whether turn-taking pauses align with detected gaze changes, requires stitching together tools that were not designed to interoperate.

VideoAnnotator addresses the pipeline integration gap; Video Annotation Viewer addresses the cross-modal inspection gap. Critically, both run entirely locally, supporting the data-privacy requirements of research involving children, patients, and recordings collected in low- and middle-income settings.

State of the field

Manual annotation platforms such as ELAN (Wittenburg et al., 2006), Datavyu (Team, 2014), and BORIS (Friard & Gamba, 2016) provide rich time-aligned coding environments but require trained human coders and do not scale to large video corpora. They are built around the assumption that a human is the annotation source, not a consumer of machine output.

Specialized computer-vision libraries such as DeepLabCut (Mathis et al., 2018) and YOLO (Jocher & Qiu, 2024) offer state-of-the-art pose estimation and detection but address a single modality and leave output standardization and batch orchestration to the user. For facial affect, Py-Feat (Cheong et al., 2023) and OpenFace (Hu et al., 2025) extract action units and emotion labels; for audio, openSMILE (Eyben et al., 2010) provides acoustic feature extraction. These are powerful tools, but each produces its own schema and none orchestrates across modalities.

General scientific visualization environments such as napari (Sofroniew et al., 2022) and FIJI/ImageJ (Schindelin et al., 2012) support extensible, multi-channel display of time-series data and are widely used in bio-imaging. They are capable tools, but their plugin ecosystems are built for image and microscopy data: they do not natively parse audio or speech-specific formats (WebVTT, RTTM). Their design assumes an interactive editing workflow rather than a read-only audit of machine-generated output.

Dataset curation platforms such as FiftyOne (Voxel51, 2020) and Label Studio (Tkachenko et al., 2020) are powerful for managing datasets and producing annotations at scale, but they are optimized for labeling pipelines rather than lightweight time-synchronized review of heterogeneous outputs.

As researchers studying parent-child interaction we needed person tracking, facial expression analysis, speech segmentation, and speaker diarization on the same set of videos. Using existing tools required ad-hoc use of multiple libraries that had no shared output format, no batch orchestration, and no unified review surface. VideoAnnotator and Video Annotation Viewer were built to close that gap, and are shared to benefit other behavioral researchers with similar requirements.

84 Software design

85 VideoAnnotator

86 VideoAnnotator is organized around four layers.

87 **Pipeline registry.** Pipelines are registered via declarative YAML metadata files. Adding a
88 new detector requires only a metadata file and a Python class inheriting from `BasePipeline`,
89 which enforces a uniform interface (`initialize()`, `process()`, `cleanup()`, `get_schema()`).
90 This metadata-driven approach allows plug-and-play composition without modifying existing
91 pipelines.

92 **Standardized output and provenance.** All outputs map to established formats: COCO JSON
93 for spatial annotations, RTTM for diarization, WebVTT for timed text. Each result is wrapped
94 with provenance metadata — pipeline name, version, configuration parameters, processing
95 timestamps — supporting reproducible re-analysis.

96 **Batch orchestration.** A thread-pool orchestrator manages concurrent job execution with
97 configurable retry strategies (fixed, linear, or exponential backoff) and transient-versus-
98 permanent error classification. Jobs are checkpointed to the local filesystem after each
99 stage, providing recovery without an external message queue.

100 **Service and CLI.** A FastAPI service exposes endpoints for job submission, status polling,
101 pipeline discovery, and results retrieval; a Typer-based CLI provides equivalent local-execution
102 commands. Both share the same orchestrator and storage backend.

103 Video Annotation Viewer

104 VAV is a stateless React/TypeScript single-page application organized around three concerns.
105 **Format parsers** validate and normalize COCO JSON, WebVTT, and RTTM files into internal
106 typed representations. **Viewer components** render a video player with overlay layer and a
107 coordinated multi-track timeline beneath it, with each annotation stream displayed as a parallel
108 lane synchronized to playback. **An optional API client** connects to a VideoAnnotator service
109 for job management and result retrieval, but is not required for standalone use. Because VAV
110 is stateless, a given video and set of annotation files deterministically produce the same display,
111 supporting reproducible review (Figure 1).

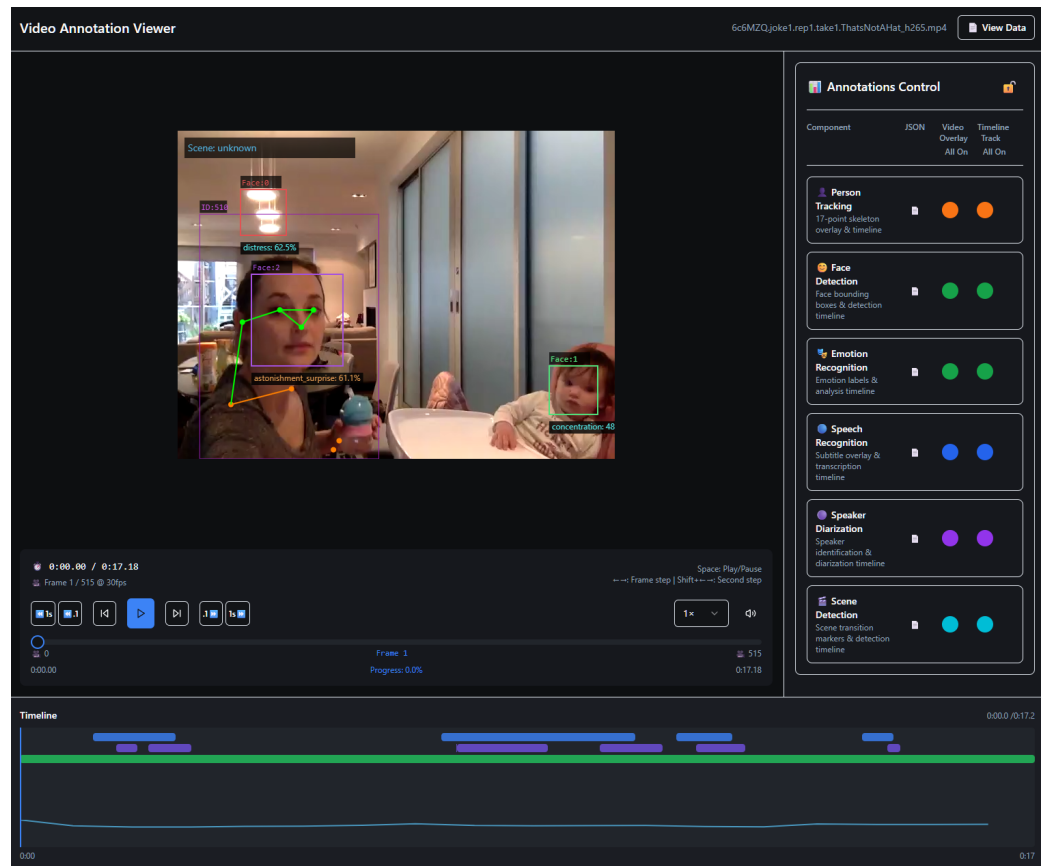


Figure 1: Video Annotation Viewer displaying a caregiver–child interaction clip with synchronized person tracking, face detection, emotion recognition, speech recognition, speaker diarization, and scene detection overlays, alongside the multi-track timeline that enables cross-modal audit.

Research impact statement

Both tools were developed at Stellenbosch University to support analysis of caregiver–child interaction videos collected across multiple sites in sub-Saharan Africa as part of the Global Parenting Initiative. Researchers at Stellenbosch University and Goldsmiths, University of London use the full pipeline — VideoAnnotator for processing and VAV for review — in developmental psychology and parenting-intervention studies. The local-first design was motivated by the ethical and governance requirements of working with video recordings of children in low- and middle-income settings. Pilot analyses have been conducted on corpora of over 500 sessions, and the software is being prepared for use in upcoming multi-site trials. A practical motivating example: VAV's synchronized multi-track timeline makes turn-taking patterns in caregiver–child interaction visible at a glance, a cue to reciprocity that is important for developmental assessment but difficult to judge from tabular outputs alone.

Quality control

VideoAnnotator maintains a pytest-based test suite covering unit, integration, and performance tests across 74 test files. Continuous integration via GitHub Actions runs tests on Ubuntu, Windows, and macOS with Python 3.12, alongside ruff linting, mypy type checking, and Trivy security scanning. Docker images provide consistent execution environments. Video Annotation Viewer is tested with Vitest and React Testing Library and built via a typed TypeScript compilation step that catches interface errors at build time.

Statement of limitations

Annotation quality is bounded by the accuracy of upstream detectors and their generalization to a given domain, population, and recording context. VideoAnnotator is a monolithic container application designed for single-machine execution and does not currently support distributed scheduling; a GPU is recommended for long videos. A more modular plugin-based design is planned for a future release. Ethical deployment — consent, data governance, and redaction — remains the responsibility of adopters. Video Annotation Viewer is a review tool and does not support editing or export of corrected annotations.

AI usage disclosure

Generative AI tools (GitHub Copilot and Claude, Anthropic) were used during development for code scaffolding, test generation, and documentation drafting. All AI-assisted outputs were reviewed, edited, and validated by the human authors, who made all core design decisions. Architectural decisions — including the pipeline registry design, the provenance-metadata format, and the modularity plan described in this paper's Software design section — were specified by the human authors as versioned, dated design documents *before* implementation (e.g. `specs/003-modular-pipeline-architecture/spec.md` and the `docs/development/roadmap_*.md` series in the repository), giving reviewers a reviewable record of design intent that is separate from, and predates, any AI-assisted code generation. No AI tools were used to generate the scientific content or analysis reported in this manuscript. All authors take full responsibility for the software and this paper.

Acknowledgements

We acknowledge the open-source communities behind the upstream models integrated by VideoAnnotator, in particular OpenFace, Whisper, pyannote, Ultralytics YOLO, LAION, and PySceneDetect.

This project was supported by the Global Parenting Initiative and funded by The LEGO Foundation.

References

- Bakeman, R., & Gottman, J. M. (1997). *Systematic observation: A strategy for counting and recording behavior* (2nd ed.). Cambridge University Press. <https://doi.org/10.1017/CBO9780511527906>
- Bredin, H., & Laurent, A. (2023). Powering speaker diarization with multi-scale neural network architecture. *Proceedings of the 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2023)*, 1–5. <https://doi.org/10.1109/ICASSP49357.2023.10096857>
- Castellano, B. (2024). *PySceneDetect: Scene change detection in videos* (Version 0.6.5). <https://www.scsedetect.com/>
- Cheong, J. H., Xie, T., Byrber, S., & Chang, L. J. (2023). Py-feat: Python facial expression analysis toolbox. *Affective Science*, 4, 781–796. <https://doi.org/10.1007/s42761-023-00191-4>
- Eyben, F., Wöllmer, M., & Schuller, B. (2010). openSMILE – the munich versatile and fast open-source audio feature extractor. *Proceedings of the 18th ACM International Conference on Multimedia*, 1459–1462. <https://doi.org/10.1145/1873951.1874246>

- 173 Friard, O., & Gamba, M. (2016). BORIS: A free, versatile open-source event-logging software
174 for video/audio coding and live observations. *Methods in Ecology and Evolution*, 7(11),
175 1325–1330. <https://doi.org/10.1111/2041-210X.12584>
- 176 Hu, J., Mathur, L., Liang, P. P., & Morency, L.-P. (2025). OpenFace 3.0: A
177 Lightweight Multitask System for Comprehensive Facial Behavior Analysis. *IEEE*
178 *International Conference on Automatic Face and Gesture Recognition (FG 2025)*.
179 <https://doi.org/10.48550/arXiv.2506.02891>
- 180 Jocher, G., & Qiu, J. (2024). *Ultralytics YOLO11* (Version 8.3.0). [https://doi.org/10.5281/](https://doi.org/10.5281/zenodo.3908559)
181 [zenodo.3908559](https://doi.org/10.5281/zenodo.3908559)
- 182 Mathis, A., Mamidanna, P., Cury, K. M., Abe, T., Murthy, V. N., Mathis, M. W., & Bethge,
183 M. (2018). DeepLabCut: Markerless pose estimation of user-defined body parts with deep
184 learning. *Nature Neuroscience*, 21(9), 1281–1289. [https://doi.org/10.1038/s41593-018-](https://doi.org/10.1038/s41593-018-0209-y)
185 [0209-y](https://doi.org/10.1038/s41593-018-0209-y)
- 186 Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A.,
187 Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual
188 models from natural language supervision. *Proceedings of the 38th International Conference*
189 *on Machine Learning*, 139, 8748–8763. <https://doi.org/10.48550/arXiv.2103.00020>
- 190 Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust
191 speech recognition via large-scale weak supervision. *Proceedings of the 40th International*
192 *Conference on Machine Learning*, 202, 28492–28518. [https://doi.org/10.48550/arXiv.](https://doi.org/10.48550/arXiv.2212.04356)
193 [2212.04356](https://doi.org/10.48550/arXiv.2212.04356)
- 194 Ramírez, S. (2018). *FastAPI* (Version 0.115). <https://fastapi.tiangolo.com/>
- 195 Schindelin, J., Arganda-Carreras, I., Frise, E., Kaynig, V., Longair, M., Pietzsch, T., Preibisch,
196 S., Rueden, C., Saalfeld, S., Schmid, B., & others. (2012). Fiji: An open-source platform
197 for biological-image analysis. *Nature Methods*, 9(7), 676–682. [https://doi.org/10.1038/](https://doi.org/10.1038/nmeth.2019)
198 [nmeth.2019](https://doi.org/10.1038/nmeth.2019)
- 199 Schuhmann, C., Kaczmarczyk, R., Rabby, G., Friedrich, F., Kraus, M., Nadi, K., Nguyen,
200 H., Kersting, K., & Auer, S. (2025). EmoNet-Voice: A Fine-Grained, Expert-Verified
201 Benchmark for Speech Emotion Detection. *arXiv Preprint arXiv:2506.09827*. <https://doi.org/10.48550/arXiv.2506.09827>
202 <https://doi.org/10.48550/arXiv.2506.09827>
- 203 Schuhmann, C., Kaczmarczyk, R., Rabby, G., Kraus, M., Friedrich, F., Nguyen, H., Kalyan, K.,
204 Nadi, K., Kersting, K., & Auer, S. (2025). EmoNet-Face: An Expert-Annotated Benchmark
205 for Synthetic Emotion Recognition. *NeurIPS*. <https://doi.org/10.48550/arXiv.2505.20033>
- 206 Serengil, S. I., & Ozpinar, A. (2020). LightFace: A hybrid deep face recognition framework.
207 *2020 Innovations in Intelligent Systems and Applications Conference (ASYU)*, 23–27.
208 <https://doi.org/10.1109/ASYU50717.2020.9259802>
- 209 Sofroniew, N., Lambert, T., Evans, K., Nunez-Iglesias, J., Bokota, G., Winston, P., Peña-
210 Castillo, L., Yamauchi, K., Bussonnier, M., Doncila Pop, D., & others. (2022). Napari:
211 A multi-dimensional image viewer for python. *Zenodo*. [https://doi.org/10.5281/zenodo.](https://doi.org/10.5281/zenodo.3555620)
212 [3555620](https://doi.org/10.5281/zenodo.3555620)
- 213 Team, D. (2014). *Datavyu: A video coding tool*. <https://datavyu.org/>
- 214 Tkachenko, M., Malyuk, M., Holmanyuk, A., & Liubimov, N. (2020). *Label studio: Data*
215 *labeling software* (Version 1.11). <https://github.com/HumanSignal/label-studio>
- 216 Voxel51. (2020). *FiftyOne* (Version 0.25). <https://github.com/voxel51/fiftyone>
- 217 Wittenburg, P., Brugman, H., Russel, A., Klassmann, A., & Sloetjes, H. (2006). ELAN
218 – linguistic annotator. *Proceedings of the Fifth International Conference on Language*
219 *Resources and Evaluation (LREC 2006)*, 1556–1559. <https://archive.mpi.nl/tla/elan>

- ²²⁰ Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Ze, Y., Yuan, C., Luo, P., Liu, W., & Wang,
²²¹ X. (2022). ByteTrack: Multi-object tracking by associating every detection box. *Computer*
²²² *Vision – ECCV 2022*, 1–21. https://doi.org/10.1007/978-3-031-20047-2_1

DRAFT